

Describe your overall approach to implementing the game

The implementation phase of *Vault Escape* involved translating our initial design into functional Java code. We followed our initial UML diagrams and started implementing our classes. This phase required us to implement core game logic, including player movement, reward collection, enemy interactions, and game state management. The focus of our approach is to create a modular and maintainable codebase, ensuring that each class has a single responsibility in accordance with object-oriented design principles.

State and justify the adjustments and modifications to the initial design of the project (shown in class diagrams and use cases from Phase 1).

As we transitioned from design to implementation, several adjustments were made to our initial design based on practical considerations and feedback from team members:

1. Class Structure Adjustment

- Specifically, we incorporated cardinality and visibility into our UML diagrams to enhance clarity and understanding of the relationships between classes.

2. Gameplay Mechanics Enhancements:

- Enhanced the enemy AI behavior for dogs, allowing them to chase the player when within a specific proximity, adding a layer of strategy and tension to gameplay.
- Modified scoring penalties to be more significant upon encountering enemies, emphasizing the importance of stealth in the game.

Explain the management process of this phase and the division of roles and responsibilities.

We adopted the **scrum** methodology for our management process, emphasizing collaborative teamwork and iterative progress. Key elements of our approach included:

- **Equal Work Distribution:** All team members participated equally in coding, testing, and reviewing. This collaborative effort helped ensure that everyone had a thorough understanding of the entire codebase.
- **Daily Check-Ins:** We held daily stand-up meetings to discuss progress, share any obstacles encountered, and plan the day's tasks. This practice fostered transparency and accountability within the team.

- **Sprints:** We organized our work based on the course deadlines and overall school schedule, each with specific goals and deliverables. At the end of each sprint, we reviewed our progress and adjusted our plans as needed.

This management process facilitated effective communication, encouraged collective problem-solving, and enabled us to respond quickly to challenges.

List external libraries you used, for instance for the GUI and briefly justify the reason(s) for choosing the libraries.

We incorporated several external libraries to facilitate our development process:

- **JavaFX:** Used for building the graphical user interface. It provides robust support for UI components and integrates seamlessly with Java applications, enhancing the visual appeal and usability of the game.
- **JUnit:** For unit testing our code. This library was chosen due to its widespread use and compatibility with Java, allowing us to implement a comprehensive testing framework easily e.g. assertion

Describe the measures you took to enhance the quality of your code

To ensure high-quality code throughout the implementation, we adopted several practices:

- Conducted regular code reviews to identify potential issues early and maintain consistent coding standards.
- Utilized JUnit for systematic testing of game functionalities, ensuring each component worked as intended before integration.
- Employed JavaDoc comments and descriptive inline comments throughout the codebase to improve readability and maintainability.

Discuss the biggest challenges you faced during this phase

- To be implemented after half-way deadline